

---

# **FACHPRAKTIKUM LAB PROJECT**

University of Stuttgart

## **GUI for Interactive Visualization of UnfoldSim**

Inderjeet Kaur Rooprai

Parth Zope

Varad Khanvilkar

Mentor: Judith Schepers

Professor: Dr. Benedikt Ehinger

GitHub Repository: [Link](#) 

---



---

## Table of Contents

|                                                              |           |
|--------------------------------------------------------------|-----------|
| <b>1. Introduction</b>                                       | <b>3</b>  |
| 1.1 Project Context                                          | 3         |
| 1.2 Problem Statement                                        | 3         |
| 1.3 Project Objectives                                       | 3         |
| 1.4 Project Timeline (Gantt Chart)                           | 4         |
| <b>2. Motivation &amp; Literature Review</b>                 | <b>4</b>  |
| 2.1 Why Build a GUI?                                         | 4         |
| 2.2 Why a GUI is Better Than a Script-Based Workflow         | 4         |
| 2.3 Literature Review                                        | 5         |
| <b>3. Requirements Identification</b>                        | <b>5</b>  |
| 3.1 How Requirements Were Identified                         | 5         |
| 3.2 Functional Requirements                                  | 5         |
| 3.3 Non-Functional Requirements                              | 6         |
| <b>4. Design: Framework Selection &amp; GUI Architecture</b> | <b>6</b>  |
| 4.1 Why the Packages Were Chosen                             | 6         |
| <b>5. Implementation &amp; Features Overview</b>             | <b>8</b>  |
| 5.1 Working Model                                            | 8         |
| 5.2 Feature Walkthrough (GUI Sequence)                       | 8         |
| Section 0: Event Variable Definition                         | 8         |
| Section 1: Design Configuration                              | 8         |
| Section 2: Component Configuration                           | 8         |
| Section 3: Onset Configuration                               | 9         |
| Section 4: Noise Configuration                               | 9         |
| Section 5: Data Management                                   | 9         |
| Main Plot and Visualization                                  | 9         |
| ERP Explorer Panel and Topoplots                             | 9         |
| Screenshots of outputs                                       | 10        |
| <b>6. Testing &amp; Validation</b>                           | <b>12</b> |
| 6.1 Testing Strategy                                         | 12        |
| <b>7. Current Limitations and Improvements</b>               | <b>12</b> |
| 7.1 Current Limitations                                      | 12        |
| 7.2 How It Could Be Improved                                 | 12        |
| <b>8. Conclusion</b>                                         | <b>13</b> |
| <b>9. References</b>                                         | <b>13</b> |

---

---

# 1. Introduction

## 1.1 Project Context



Electroencephalography (EEG) is one of the most widely used non-invasive techniques for studying the human brain. Within experimental psychology and cognitive neuroscience, researchers frequently rely on Event-Related Potentials (ERPs) time locked brain responses to stimuli to study perception, cognition, and attention. However, before experiments are conducted on real participants, simulation frameworks allow researchers to test and validate their analysis pipelines in simulations, using known ground-truth signals.

This project was undertaken as part of the Fachpraktikum at the University of Stuttgart. The goal was to develop an interactive, browser based Graphical User Interface (GUI) for the `UnfoldSim.jl` ecosystem a Julia based ERP simulation package developed by the `Unfold Toolbox` group. The resulting application enables researchers to `configure, simulate, and visualize EEG/ERP signals` interactively without writing a single line of code.

## 1.2 Problem Statement

EEG researchers often need to create simulated brain signal data whether to test their analysis methods, check that their pipelines work correctly, or build examples for teaching. Right now, doing this with `UnfoldSim.jl` means writing Julia code every single time you want to change something. That makes it complex.

The current workflow has a few difficulties:

- `Want to change a parameter? You're back to editing code.`
- `No live preview you run the script, wait, and hope it looks right.`
- Combining multiple signal components (like N170 and P300 together) gets complicated with no visual guide to help.
- You have to learn the entire API just to run your first basic simulation.



All of this `slows researchers down` and puts the tool out of reach for students or anyone just getting started. A simple, visual interface that handles the complexity behind the scenes while still giving advanced users full control would make `the whole thing` a lot more approachable.

## 1.3 Project Objectives

The primary objectives of this project were:

- Design and implement a fully reactive, web-based GUI for `UnfoldSim.jl` using the `Bonito.jl` and `WGLMakie.jl` frameworks in Julia.
- Support multiple simulation model types: Linear Model, Mixed Model, and Multi-channel Model.
- Enable the definition of custom event variables (both categorical and continuous) with formula-driven amplitude modulation.
- Integrate `HArtMuT` a realistic head and source model for multi-channel EEG topoplot visualization with 227 electrodes.
- Provide heatmap topoplots (amplitude) using the `TopoPlots.jl` package.
- Implement robust data management: JSON configuration download/upload, CSV export, and JLD2 simulation object saving.
- Ensure the interface is usable on both dark and light themes and handles edge cases gracefully.



## 1.4 Project Timeline (Gantt Chart)

| Project Phase                         | Oct | Nov | Dec | Jan | Feb | Mar | Dates           |
|---------------------------------------|-----|-----|-----|-----|-----|-----|-----------------|
| Project Setup & Literature Review     |     |     |     |     |     |     | Oct 13 – Oct 24 |
| Requirements & Feature Identification |     |     |     |     |     |     | Oct 28 – Nov 09 |
| GUI Design (Prototyping)              |     |     |     |     |     |     | Nov 10 – Nov 30 |
| GUI Implementation – Phase 1          |     |     |     |     |     |     | Dec 01 – Jan 05 |
| GUI Implementation – Phase 2          |     |     |     |     |     |     | Jan 06 – Feb 02 |
| Stretch Goals Development             |     |     |     |     |     |     | Feb 03 – Feb 23 |
| Testing & Refinement                  |     |     |     |     |     |     | Feb 24 – Mar 17 |
| Documentation & Final Report          |     |     |     |     |     |     | Mar 18 – Mar 31 |

Table 1: Gantt Chart

## 2. Motivation & Literature Review

### 2.1 Why Build a GUI?

Scientific simulation tools are typically built for programmers, but the people who need them are far more diverse students, junior researchers, experienced scientists, and clinicians. Not everyone needs to learn a programming language just to run a simulation. Even for experts who can write scripts, a GUI offers something scripts cannot: instant feedback. Moving a slider and immediately seeing how noise affects a waveform builds intuition far faster than editing code and re-running it. This makes exploration quicker and results easier to interpret for everyone, regardless of their programming background. The GUI was built to make a powerful simulation tool accessible to anyone who needs it not just those who can code.

### 2.2 Why a GUI is Better Than a Script-Based Workflow

The following comparison highlights the practical advantages of the GUI approach:

- Immediate feedback: Slider adjustments trigger simulation re-runs within 500ms, versus minutes of code-edit-run cycles.
- Discoverability: Dropdown menus expose all available onset types and noise models, removing the need to consult documentation for every option.
- Error prevention: The GUI prevents invalid combinations (e.g., Mixed Model with Single-subject design) with clear user messaging, whereas scripts silently fail or throw cryptic Julia errors.
- Reproducibility: Configurations are exportable as JSON and CSV with embedded metadata, enabling one-click restoration of any prior experiment state.
- Accessibility: Non-Julia users (e.g., Python-fluent neuroscientists) can use the tool through any browser without installing or learning Julia syntax.

## 2.3 Literature Review

### UnfoldSim.jl - (UnfoldSim Homepage)

UnfoldSim.jl (Schepers et al., 2024) is a Julia package for simulating continuous EEG with overlapping ERP responses. It implements a four-stage design-component-onset-noise simulation pipeline. The design stage specifies the experimental structure (conditions, subjects, items) using SingleSubjectDesign, RepeatDesign, or MultiSubjectDesign. The component stage defines the ERP shape via a basis function, regression formula, and beta coefficients, supporting both LinearModelComponent and MixedModelComponent for fixed and random effects respectively. The onset stage controls inter-stimulus timing using UniformOnset, LogNormalOnset, or NoOnset. The noise stage adds realistic signal contamination via WhiteNoise, PinkNoise, RedNoise, or NoNoise. The package integrates with the HArtMuT head model for realistic multi-channel source projection.

### Bonito.jl and Reactive Web GUIs in Julia

Bonito.jl is a Julia web framework built on top of WGLMakie that enables server-side reactive applications delivered to the browser. It uses an Observable-based reactivity model: Observables hold values and automatically trigger downstream computations when updated, making it natural to build live-updating simulation dashboards. Alternative Julia GUI options including Interact.jl (deprecated) and Dash.jl (limited WGLMakie integration) were evaluated and rejected in favour of Bonito.jl.

## 3. Requirements Identification

### 3.1 How Requirements Were Identified

**Iterative prototyping:** We built simple test versions of the GUI starting with rough sketches on paper and a basic working prototype using Bonito. This early testing helped us discover important technical challenges we hadn't anticipated, such as how to prevent the app from slowing down when sliders are moved too quickly, how to handle multiple buttons clicks happening at the same time, and how to correctly count terms in a formula. Identifying these issues early on shaped the technical requirements for the rest of the project.

### 3.2 Functional Requirement

| ID   | Requirement           | Description                                                                             |
|------|-----------------------|-----------------------------------------------------------------------------------------|
| FR-1 | Real-time Simulation  | Auto-triggers UnfoldSim.jl on slider/dropdown changes with 500ms debounce throttling.   |
| FR-2 | Component Tabs        | Multiple ERP tabs (N170, P300, P100, N400, Custom) with cumulative signal overlay.      |
| FR-3 | Event Variable System | Define categorical and continuous event variables via templates or custom input fields. |
| FR-4 | Design Types          | Single-subject, Repeat, and Multi-subject designs with context-appropriate parameters.  |
| FR-5 | Onset Distributions   | No Onset, Uniform, or LogNormal with configurable mu, sigma, offset, and truncation.    |
| FR-6 | Noise Models          | White, Pink, Red noise and No Noise with adjustable amplitude level slider.             |
| FR-7 | Multi-channel EEG     | HArtMuT 227-electrode model with amplitude and RMS topoplot heatmaps.                   |

| ID    | Requirement          | Description                                                                   |
|-------|----------------------|-------------------------------------------------------------------------------|
| FR-8  | Data Export          | Download config as JSON, simulation data+events as CSV, full object as .jld2. |
| FR-9  | Configuration Upload | Re-upload previously saved CSV to restore all slider and dropdown states.     |
| FR-10 | Theme Toggle         | Dark/light mode switch with full CSS variable system for accessibility.       |

Table 2: Functional Requirements

### 3.3 Non-Functional Requirements

| ID   | Category        | Requirement                                                                           |
|------|-----------------|---------------------------------------------------------------------------------------|
| NF-1 | Performance     | Slider updates debounced at 500ms; time slider throttled at 50ms to prevent overload. |
| NF-2 | Usability       | All controls labeled with live-updating value display and formula hint boxes.         |
| NF-3 | Reliability     | Mutex/generation locks prevent race conditions on multi-electrode click events.       |
| NF-4 | Maintainability | Modular structure: ERPEXplorer submodule, ComponentTab struct, Observable reactivity. |
| NF-5 | Portability     | Browser-based via Bonito.jl; accessible on any machine with Julia installed.          |
| NF-6 | Compatibility   | Works with UnfoldSim.jl, WGLMakie, TopoPlots.jl, JLD2, CSV, JSON3.                    |

Table 3: Non-Functional Requirements

## 4. Design: Framework Selection & GUI Architecture

### 4.1 Why the Packages Were Chosen

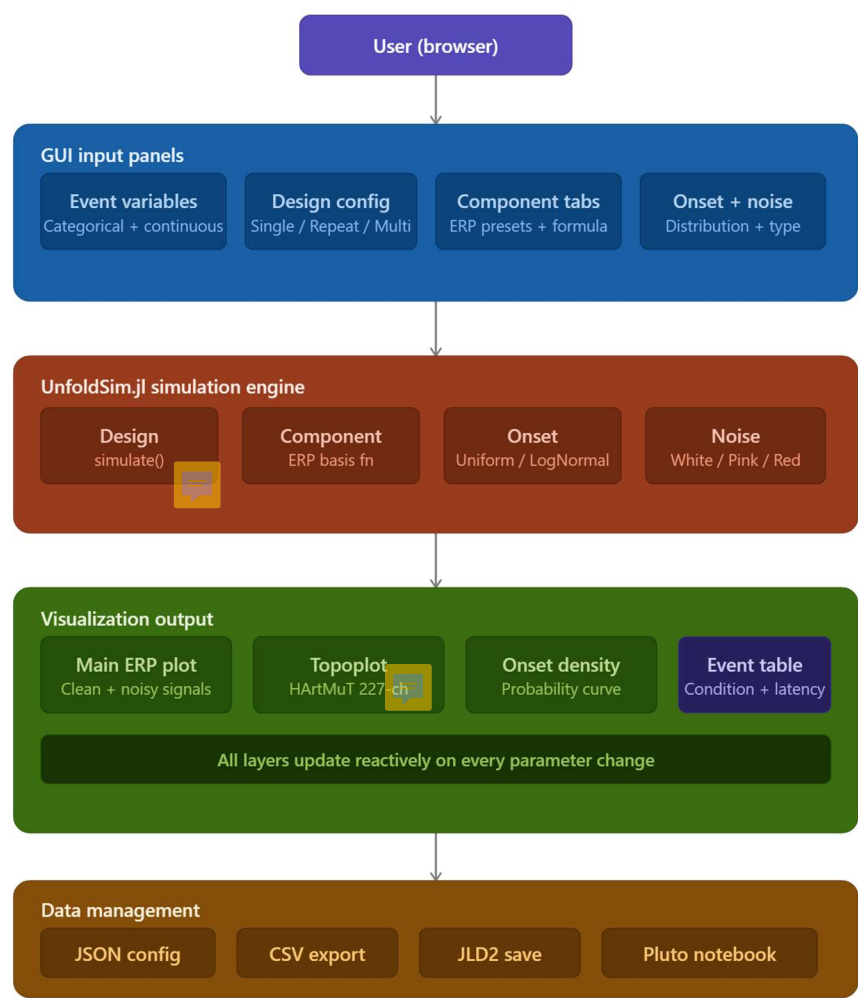
Package selection was driven by three criteria: native Julia integration (to avoid inter-language marshalling overhead for large EEG matrices), reactive programming support (to enable live updates without manual refresh), and compatibility with UnfoldSim.jl data structures. The table below summarises all selected packages and their justifications:

| Package           | Role                  | Justification                                                               |
|-------------------|-----------------------|-----------------------------------------------------------------------------|
| Bonito.jl         | Web GUI framework     | Reactive DOM + Observable system; serves interactive HTML in browser.       |
| WGLMakie.jl       | Plotting backend      | GPU-accelerated, interactive plots embedded in the Bonito session.          |
| UnfoldSim.jl      | EEG simulation engine | Core library; provides design, component, onset and noise APIs.             |
| TopoPlots.jl      | Topoplot rendering    | Dedicated EEG scalp topography with Delaunay interpolation and colorbar.    |
| HArtMuT           | Head model            | 227-channel realistic head model for multi-channel EEG scalp projection.    |
| JLD2.jl           | Binary serialization  | HDF5-compatible Julia serialization for full Simulation struct persistence. |
| CSV.jl / JSON3.jl | Text I/O              | Fast CSV and JSON export/import for data and configuration files.           |

| Package          | Role                      | Justification                                                                 |
|------------------|---------------------------|-------------------------------------------------------------------------------|
| Observables.jl   | Reactive state            | Publisher-subscriber model; all GUI state flows through Observable nodes.     |
| DataFrames.jl    | Event table management    | Stores and displays trial-by-trial event data with conditions and latencies.  |
| Distributions.jl | Probability distributions | Provides Log-Normal distribution used for realistic onset jitter modeling.    |
| ColorSchemes.jl  | Color mapping             | Perceptually uniform color maps for multi-channel and topoplot visualization. |

Table 4: Package Description

Architecture Diagram:



---

## 5. Implementation & Features Overview

### 5.1 Working Model

The implemented GUI is a fully functional web application served via Bonito.jl that runs in any modern browser. The simulation backend leverages UnfoldSim.jl's `simulate()` function, which accepts a Design, one or more Component objects, an Onset distribution, and a Noise model. The reactive pipeline ensures that every parameter change triggers a new simulation after a configurable debounce period (0.5 seconds by default for the main simulation, 0.05 seconds for the time-slider topoplot update). On startup, the HArtMuT head model is loaded once and cached globally (CACHED\_HARTMUT) to avoid the significant initialization overhead on subsequent simulations. Electrode positions are also pre-computed and cached as a dictionary mapping electrode symbol to normalized 2D coordinates. The GUI initializes with a default 'Custom' component tab and a single-subject design with condition = [A, B], allowing immediate exploration without any setup.

### 5.2 Feature Walkthrough (GUI Sequence)



#### Section 0: Event Variable Definition

The Event Definition panel is the starting point of every simulation, where users define their experimental conditions. Categorical variables (condition [A, B], stimulus\_type, task, emotion, color) and continuous variables (contrast, intensity, duration, frequency) can be added from five built-in templates or defined freely using custom names, levels, and value ranges. Every change instantly updates the categorical\_variables and continuous\_variables Observables, which feed into the design builder to construct the correct SingleSubjectDesign, RepeatDesign, or MultiSubjectDesign, and into the formula term counter to correctly size the beta vector. A formula dropdown below the panel allows users to select a formula directly, with all available formulas pre-define based on the current event variables. The current status variable state is always visible in a live-updated display at the bottom of the panel.

#### Section 1: Design Configuration

The Design Configuration panel allows selection of the experimental design type. Three options are available. Single-subject design produces data from a single participant with all defined conditions. Repeat design repeats a single-subject design a specified number of times, useful for generating trial sequences with many repetitions. Multi-subject design uses MultiSubjectDesign, which assigns items between subjects according to the first categorical variable. The number of subjects and items are controlled by dedicated sliders with mapped values. When the model type is set to Mixed Model, the design is automatically forced to Multi-subject design because MixedModelComponent requires subject-level random effects. An informational warning is displayed when this constraint is active. For Linear Model with Multi-subject design, the code automatically wraps the formula in a mixed-effects structure with zero random variance ( $\sigma = 0$ ), enabling within-design subject boundaries to be displayed on the main plot.

#### Section 2: Component Configuration

ERP components are managed through a tabbed interface. Each tab encapsulates: model type (Linear, Mixed, or Multi-channel); Hanning preset (N170, N400, P100, P300, Custom) that auto-populates the basis function field; an editable basis function text field; a StatsModels.jl formula string referencing the user-defined event variables; DummyCoding or EffectsCoding selection; and beta, contrast, and sigma sliders, each with an input box for entering values directly. A Custom tab allows users to add any Hanning preset manually, and all currently active tabs are displayed and visible at all times. Tabs can also be removed at any point using the Remove Tab button. Multiple tabs are active simultaneously; their clean signals are summed by compute\_cumulative\_signal to produce the cumulative overlay visible on the main plot.

---



---

### Section 3: Onset Configuration

The onset dropdown lets you choose how events are timed in the simulation. Selecting No Onset produces evenly spaced events with no timing variation. Uniform adds a random delay within a fixed range, controlled by two sliders (width and offset). Log-Normal models more realistic, uneven timing using five sliders (mean, spread, offset, and two cut-off limits). Whichever option is selected, a live density curve updates instantly in the lower panel, giving a visual preview of how events will be spread across time.

### Section 4: Noise Configuration

Four noise types are available: No Noise (clean signal only), White Noise (flat power spectrum), Pink Noise ( $1/f$  power spectrum, characteristic of EEG), and Red Noise ( $1/f^2$ , Brownian motion-like). Each noise type is instantiated from UnfoldSim's noise module with a configurable noise level, which is controlled by a dedicated slider mapping 0-100 to noise amplitude 0.0-10.0.

### Section 5: Data Management

Four data management options are available. First, Download Configuration saves all 22 current slider and dropdown values as a JSON file to the browser's download folder. Second, Download Simulation Object exports the multichannel EEG data and event table as CSV files, with the configuration settings embedded inside the events file for reference. Third, Save as JLD2 saves the complete simulation including clean and noisy EEG data, events, time vector, and settings as a timestamped .jld2 file that can be reloaded in any Julia environment to re-run or analyze the simulation directly. Fourth, Upload Simulation allows a previously saved CSV to be re-uploaded; the GUI reads the embedded configuration from the file and automatically restores all settings, triggering a new simulation that reproduces the original result exactly. A Python script was developed to convert the downloaded JSON configuration file into ready-to-run Julia code, formatted as a Pluto notebook. This allows users to take any simulation they configured in the GUI and instantly reproduce it as a fully executable Julia script, without manually translating the settings into code. The generated notebook includes all parameter values, model types, and formula definitions, making it a practical bridge between the visual GUI workflow and a code-based research environment.

### Main Plot and Visualization

The center panel contains two plots. The main plot displays multiple overlapping lines: the cumulative noisy signal (gray), cumulative clean signal (red), per-tab ERP component lines (dashed, color-coded), selected electrode waveforms (multi-coloured), Hanning reference waveforms for standard ERP types, and vertical markers showing event onsets per condition. Every layer updates automatically when any parameter changes. A legend below the plot clearly identifies each line by color including condition names, subject boundaries, and selected electrodes making the plot easy to read at a glance. At the bottom, a second smaller plot shows a live density curve of the current onset timing distribution, updating instantly as onset parameters are adjusted.

### ERP Explorer Panel and Topoplots

When Multi-channel Model is selected, the right panel shows three interactive visualizations. The dot map displays all 227 HArtMuT electrodes as colored dots on a 2D head outline; clicking any electrode selects it and adds its signal as an extra line on the main plot. The amplitude topoplot uses TopoPlots.jl to draw a smooth color map of scalp voltage at the current time point selected by the time slider, using a red-blue color scale where red and blue indicate opposite voltage polarities.

The event table shows a scrollable table listing every trial in the simulation, with columns for sample index, condition, subject, latency, and any custom event variables the user has defined. Condition labels are color-coded blue for the standard condition and red or green for others making it easy to scan and distinguish between different trial types at a glance.

---

Screenshots of outputs:



Figure 1: The figure demonstrates how multiple ERP components can be layered to produce a cumulative signal. A Linear Model with DummyCoding was selected, and the formula `@formula(0 ~ 1 + condition)` was applied to define the condition-based amplitude modulation. Two Hanning presets were then applied sequentially first N170 (Negative), then P300 (Positive) each appearing as individual dashed waveforms on the plot in yellow and pink respectively. The cumulative clean signal shown in red is the direct sum of both component waveforms. This shows how the GUI allows multiple ERP components to be built up one at a time, with the final plot reflecting the combined output of all active presets.

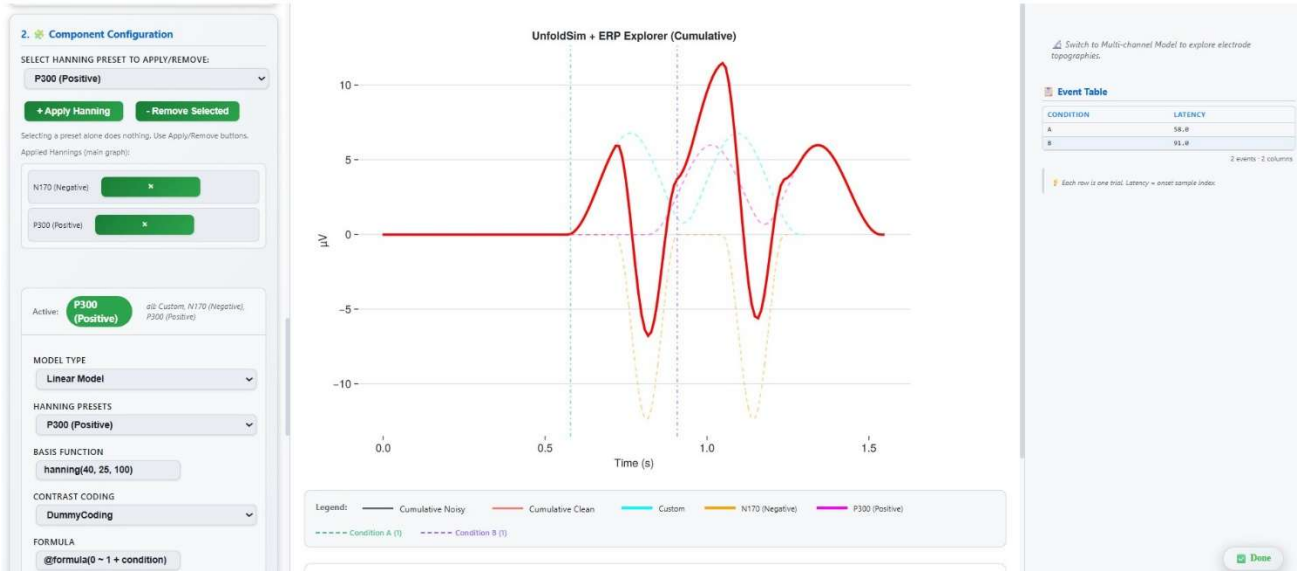


Figure 2: The figure demonstrates the multi-channel simulation capability of the GUI. A Single-subject design was selected, and the model type was set to Multi-channel Model with a Custom Hanning preset using the basis function `hanning(40, 0.5, 100)` and a projection vector of `[1.0, 0.5, -0.2]`. The main plot shows the resulting waveforms for all active electrodes simultaneously, each rendered in a different colour. On the right, the ERP Explorer panel displays the full HARTMuT 227-channel electrode map as a dot plot, and below it the Amplitude Topoplot shows the scalp voltage distribution at the current time point using a red-blue colour scale, where red and blue indicate opposite voltage polarities. The event table lists two trials for conditions A and B with their respective latencies.





Figure 3: The figure shows the Onset Configuration, Global Noise, and Data Management sections of the GUI. A Log-Normal onset distribution was selected, with the parameters set as  $\mu = 1.1$ ,  $\sigma = 0.5$ , offset = 20, truncate lower = 0.0, and truncate upper = 248. The resulting distribution is visualised in the Onset Distribution Probability Density plot at the bottom, which shows a sharp peak around the 20–25 sample range, reflecting the effect of the offset and truncation settings applied. The Global Noise section is set to No Noise, meaning the simulation runs with a clean signal only. The Data Management section provides four options Download Configuration, Download Simulation Object, Save Simulation (.jld2), and Upload Saved Simulation allowing the user to save and restore the full simulation state at any point.



---

## 6. Testing & Validation

### 6.1 Testing Strategy

Testing was conducted during the final project presentation through a structured set of hands-on test cases prepared in advance. **Participants primarily researchers and students** followed specific scenarios to interact with and evaluate the GUI directly. Test cases covered all major features, for example: creating a multi-subject design with the N170 Hanning preset applied, defining a custom categorical event variable with a matching formula, adjusting noise levels and observing the waveform response, and exporting the simulation as a JLD2 file. Each scenario was designed to verify both the correctness of the simulation output and the usability of the interface. **Feedback was collected informally during the session, with observers noting any errors, hesitations, or points of confusion, providing practical insight into the system's real-world usability.** Overall, the GUI behaved robustly across all tested scenarios.



## 7. Current Limitations and Improvements

### 7.1 Current Limitations

Despite the range of features implemented, several limitations exist in the current version:

- **Formula validation is delayed:** If a formula is typed incorrectly, the error only appears when the simulation runs, not while typing. Catching mistakes earlier would make the tool easier to use.
- **Slow initial loading:** The HArtMuT head model takes 10–15 seconds to load on first launch. Since the interface appears blank during this time, new users may think the application has frozen.
- **No session persistence:** All settings are lost when the browser tab is closed. While configurations can be saved and re-uploaded manually, there is no automatic recovery of the previous session.
- **Limited multi-subject visualisation:** Subject boundaries are marked with vertical lines on the plot, but individual subject ERPs cannot be viewed separately.
- **No continuous EEG simulation:** The current system produces fixed waveforms. Dynamically changing signals that respond to user input in real time are not yet supported.



### 7.2 How It Could Be Improved

- **Visual formula builder:** Add a drag-and-drop interface in the formula tab where users can directly drag event variables into the formula, removing the need to define or type variables manually.
- **Collaboration mode:** Allow multiple researchers to work on the same simulation simultaneously, making it more suitable for group teaching and research sessions.
- **Real-time parameter control:** Allow users to adjust simulation settings while the simulation is still running, without needing to restart it each time.
- **High-performance computing support:** Connect the system to a computing cluster so that large-scale simulations involving thousands of subjects can be run efficiently.

## 8. Conclusion

This project successfully delivered a fully functional, browser-based GUI for UnfoldSim.jl, built entirely in Julia using Bonito.jl and WGLMakie.jl, covering the complete EEG simulation pipeline from event variable definition and experimental design to noise modelling, multi-channel topoplot visualisation, and data export — all without requiring any programming knowledge. Researchers and students can now visually explore simulation parameters, instantly see their effects, and export fully reproducible configurations, making complex EEG analysis significantly more accessible. Key technical achievements include the 227-channel HArtMuT topoplot integration, multi-tab ERP component accumulation, a reactive Observable signal graph, and a complete data management pipeline with JLD2 serialization. Through this project, practical hands-on experience was gained with Julia and EEG processing, directly linking theoretical EEG concepts to a real working tool. Testing confirmed that all functional requirements were met, and the overall user experience was validated by researchers and students during the final presentation. Future work should focus on inline formula validation, compatibility with MNE-Python and EEGLAB to make the tool even more useful for the broader neuroscience community. The current implementation provides a solid, extensible foundation that showcases both technical skill and the potential for impactful, accessible EEG research tooling.

## 9. Reference

[1] CLI or GUI — Bachelor Thesis (Linköping University)



Henriksen, K., & Wille, S. (2015). *CLI or GUI: A study of preference and performance among Git users* [Bachelor thesis]. Linköping University. <https://www.diva-portal.org/smash/get/diva2:852747/FULLTEXT02>

[2] Graphical User Interfaces — Oxford HCI Textbook Chapter

Hornbæk, K., Kristensson, P. O., & Oulasvirta, A. (2025). Graphical user interfaces. In *Introduction to Human-Computer Interaction* (pp. 485–502). Oxford University Press. <https://doi.org/10.1093/oso/9780192864543.003.0028>

[3] A Command Line Interface versus a Graphical User Interface (VR Systems)

Fellmann, M., & Kavakli, M. (2007). *A command line interface versus a graphical user interface in coding VR systems*. ResearchGate. <https://www.researchgate.net/publication/234818436>

[4] Comparing Response Time, Errors, and Satisfaction — GUI vs Text Interface

Staggers, N., & Kobus, D. (2000). Comparing response time, errors, and satisfaction between text-based and graphical user interfaces during nursing order tasks. *Journal of the American Medical Informatics Association*, 7(2), 164–176. <https://doi.org/10.1136/jamia.2000.0070164>

[5] Schepers, J., Lips, L., Marathe, M., & Ehinger, B. V. (2025). UnfoldSim.jl: Simulating continuous event-based time series data for EEG and beyond. *Journal of Open Source Software*, 10(107), 6641. <https://doi.org/10.21105/joss.06641>

[6] Danisch, S., & Krumbiegel, J. (2021). Makie.jl: Flexible high-performance data visualization for Julia. *Journal of Open Source Software*, 6(65), 3349. <https://doi.org/10.21105/joss.03349> (replaces your [2] — proper peer-reviewed citation for Makie/WGLMakie)



\* We used a combination of Claude (Anthropic Sonnet 4.6) for content structuring and Quillbot for grammar and editing assistance.